

**LAPORAN HASIL SISTEM FOOD DELIVERY TERINTEGRASI
BERBASIS ARSITEKTUR MICROSERVICES DENGAN
IMPLEMENTASI GRAPHQL API GATEWAY DAN
CONTAINERIZATION**

**Studi Kasus: Ekosistem Layanan Pemesanan, Pengiriman, dan Pembayaran Digital
Lintas Platform**



**Disusun Oleh:
Kelompok Mataram Is Red**

Muhammad Rayhan Ramadhan 102022330145

Darvesh Gladwin Musyaffa 102022300082

Jehezkiel Agna Saputra 102022300416

**PROGRAM STUDI S1 SISTEM INFORMASI
FAKULTAS REKAYASA INDUSTRI
UNIVERSITAS TELKOM
BANDUNG
2025**

RINGKASAN

Proyek ini bertujuan untuk mengembangkan sistem *Food Delivery* berskala *enterprise* yang dirancang menggunakan arsitektur *Microservices* untuk memecah kompleksitas proses bisnis menjadi layanan-layanan yang otonom dan terisolasi. Sistem ini terdiri dari lima layanan utama yang saling berkolaborasi, yaitu: User Service untuk manajemen identitas multi-role (Pelanggan, Driver, Admin), Restaurant Service untuk pengelolaan katalog menu dinamis, Order Service sebagai pusat orkestrasi transaksi, Driver Service untuk manajemen armada pengiriman, dan Payment Service yang menangani transaksi keuangan.

Seluruh ekosistem ini disatukan melalui API Gateway yang mengimplementasikan GraphQL sebagai standar komunikasi utama, memberikan fleksibilitas pengambilan data yang efisien bagi aplikasi klien. Infrastruktur sistem dibangun di atas teknologi Docker, di mana setiap layanan memiliki basis data terpisah (*Database per Service*) untuk menjamin *loose coupling*. Selain fokus pada modularitas internal, sistem ini juga dirancang untuk interoperabilitas eksternal, di mana layanan pembayaran terintegrasi dengan sistem *E-Wallet* mitra (DOSWallet) untuk memvalidasi transaksi secara *real-time*, menyimulasikan lingkungan bisnis digital yang saling terhubung.

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Dalam ekosistem ekonomi digital saat ini, platform layanan pesan-antar makanan (*Food Delivery*) telah berevolusi menjadi sistem yang sangat kompleks. Tidak sekadar menghubungkan pembeli dan penjual, platform ini harus mengelola sinkronisasi data secara *real-time* antara berbagai entitas yang memiliki karakteristik berbeda: pelanggan yang membutuhkan respons cepat, restoran yang harus mengelola stok menu secara dinamis, serta mitra pengemudi yang bergerak dengan lokasi yang terus berubah. Pendekatan arsitektur monolitik tradisional, di mana seluruh logika bisnis digabungkan dalam satu basis kode, sering kali gagal menangani kompleksitas ini. Kegagalan pada satu modul, misalnya fitur pelacakan pengemudi, dapat melumpuhkan seluruh sistem pemesanan, menciptakan risiko operasional yang fatal bagi keberlangsungan bisnis.

Untuk mengatasi tantangan skalabilitas dan keandalan tersebut, penerapan arsitektur *Microservices* menjadi solusi yang mutlak diperlukan. Dalam proyek ini, sistem dipecah menjadi lima layanan spesifik: *User Service* untuk menangani keamanan dan profil pengguna; *Restaurant Service* untuk manajemen katalog makanan; *Order Service* untuk menangani siklus hidup pesanan (dari persiapan hingga pengantaran); *Payment Service* untuk memproses transaksi keuangan; dan *Driver Service* untuk manajemen alokasi pengemudi. Pemisahan ini memungkinkan setiap layanan dikembangkan, diperbarui, dan ditingkatkan kapasitasnya (*scale-out*) secara independen tanpa mengganggu layanan lain, serta memungkinkan penggunaan teknologi basis data yang paling sesuai untuk kebutuhan masing-masing layanan.

Efisiensi komunikasi data juga menjadi isu krusial dalam aplikasi yang melibatkan banyak layanan mikro. Penggunaan REST API standar sering kali menyebabkan masalah *over-fetching* (mengambil data berlebih) atau *under-fetching* (kekurangan data sehingga butuh banyak *request*), yang berdampak buruk pada performa aplikasi *mobile*. Oleh karena itu, proyek ini mengimplementasikan GraphQL sebagai lapisan komunikasi utama (*API Layer*). GraphQL memungkinkan klien untuk mendefinisikan struktur data yang spesifik dalam satu kali permintaan, secara drastis mengurangi beban jaringan dan latensi. Hal ini didukung oleh teknologi Docker, yang membungkus setiap layanan mikro ke dalam *container* terisolasi, memastikan konsistensi lingkungan dari tahap pengembangan hingga produksi serta mempermudah proses orkestrasi layanan.

Selain tantangan internal, sistem *enterprise* modern juga dituntut untuk memiliki interoperabilitas tinggi dengan sistem eksternal. Dalam skenario nyata, sebuah platform *Food*

Delivery jarang beroperasi secara tertutup; ia harus terhubung dengan ekosistem pembayaran digital yang lebih luas. Oleh karena itu, proposal ini juga mencakup implementasi integrasi lintas sistem (*Cross-System Integration*), dimana Payment Service internal akan berkomunikasi dengan layanan dompet digital eksternal (DOSWallet) melalui standar API yang disepakati. Integrasi ini tidak hanya memvalidasi status pembayaran pengguna secara *real-time*, tetapi juga membuktikan kemampuan sistem untuk berinteraksi dengan layanan pihak ketiga yang heterogen, memenuhi standar aplikasi *enterprise* yang terintegrasi penuh.

1.2 Tujuan Pembelajaran

Selaras dengan tujuan Tugas Besar mata kuliah Integrasi Aplikasi Enterprise, pengembangan sistem ini bertujuan agar mahasiswa mampu:

1. Membangun Arsitektur Modular: Merancang dan mengimplementasikan sistem aplikasi *enterprise* yang terbagi menjadi layanan-layanan mikro (*microservices*) yang otonom dan terintegrasi.
2. Deployment Berbasis Container: Menggunakan teknologi Docker untuk melakukan *containerization* pada setiap layanan (User, Order, Restaurant, Payment, Driver) beserta basis datanya masing-masing.
3. Implementasi GraphQL: Menerapkan GraphQL sebagai *API layer* untuk komunikasi data yang fleksibel dan efisien antar layanan maupun ke sisi klien.
4. Integrasi Lintas Kelompok: Membuktikan standar interoperabilitas dengan melakukan integrasi nyata (bertukar data) antara sistem internal dengan sistem eksternal (DOSWallet) milik kelompok lain.

1.3 Ruang Lingkup dan Batasan

Ruang Lingkup:

1. Pengembangan 5 layanan internal berbasis Microservices:
 - a. User Service: Manajemen autentikasi (JWT) dan profil pengguna (Customer, Driver, Admin).
 - b. Restaurant Service: Manajemen data restoran dan menu makanan.
 - c. Order Service: Orkestrasi pemesanan dan kalkulasi total biaya.
 - d. Payment Service: Simulasi pembayaran internal dan gerbang integrasi ke layanan eksternal.
 - e. Driver Service: Manajemen data pengemudi dan alokasi pesanan.
2. Penerapan Database per Service, di mana setiap layanan memiliki basis data terpisah (MySQL) untuk menjamin *loose coupling*.

3. Integrasi eksternal wajib: Menghubungkan sistem dengan Payment Service/Wallet Service milik kelompok lain (DOSWallet) via GraphQL.

Batasan:

1. Sistem pembayaran eksternal (*External Payment*) bersifat *external dependency*. Fungsi pembayaran penuh bergantung pada ketersediaan dan kontrak API dari sistem DOSWallet milik kelompok mitra.
2. Fokus utama pengembangan adalah pada sisi *backend*, arsitektur integrasi, dan API. Antarmuka pengguna (*Frontend*) dikembangkan sebagai pendukung untuk mendemonstrasikan alur integrasi, bukan fokus utama penilaian.

BAB 2 ARSITEKTUR SISTEM

2.1 Prinsip Desain

Arsitektur sistem ini dirancang dengan berlandaskan pada empat pilar utama *Microservices*:

1. Separation of Concerns: Identitas pengguna, manajemen katalog restoran, logika transaksi pemesanan, dan manajemen armada pengiriman dipisahkan secara tegas ke dalam layanan yang berbeda untuk memastikan fokus fungsional yang jelas.
2. Service Autonomy: Setiap layanan (*User, Restaurant, Order, Payment, Driver*) memiliki basis data (*database*) masing-masing yang terisolasi. Hal ini mencegah ketergantungan data langsung (*tight coupling*) dan memungkinkan setiap layanan untuk berkembang secara independen.
3. Interoperability: Seluruh komunikasi antarlayanan maupun dengan aplikasi klien menggunakan standar GraphQL. Ini menjamin kontrak data yang eksplisit, efisien (mencegah *over-fetching*), dan memudahkan integrasi lintas platform.
4. Containerized Deployment: Seluruh komponen sistem dijalankan dalam lingkungan terisolasi menggunakan Docker Container, memastikan konsistensi konfigurasi di berbagai lingkungan pengembangan dan memudahkan proses orkestrasi.

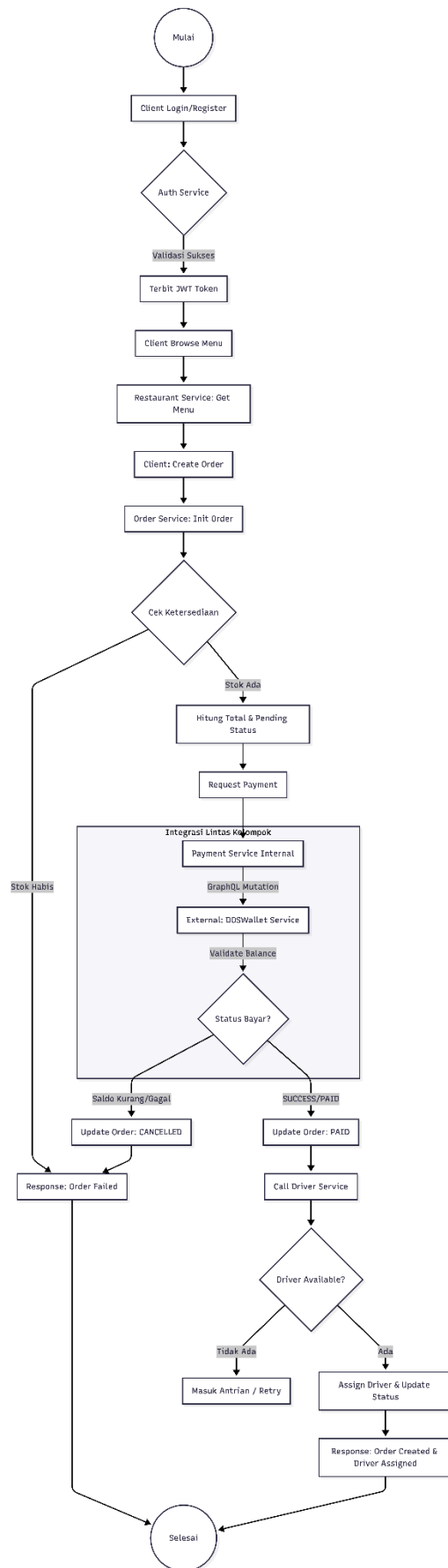
2.2 Komponen dan Tanggung Jawab

Sistem terdiri dari lima layanan mikro internal dan satu integrasi eksternal:

1. User Service (Identity Provider): Bertanggung jawab atas autentikasi (Login/Register), penerbitan token JWT, dan manajemen profil pengguna multi-role (Customer, Driver, Admin).
2. Restaurant Service (Catalog Manager): Mengelola data master restoran, daftar menu, harga, dan ketersediaan stok secara dinamis.
3. Order Service (Transaction Orchestrator): Pusat logika bisnis yang menangani siklus hidup pesanan, kalkulasi biaya total, dan mengoordinasikan komunikasi antara User, Restaurant, dan Payment.
4. Payment Service (Internal & Gateway): Mencatat riwayat transaksi internal dan bertindak sebagai gerbang (*gateway*) untuk berkomunikasi dengan sistem pembayaran eksternal.
5. Driver Service (Fleet Manager): Mengelola data pengemudi, status ketersediaan (*online/offline*), dan alokasi pengemudi untuk pesanan yang masuk.
6. External Payment Service (DOSWallet - Kelompok Mitra): Layanan dompet digital pihak ketiga yang dikonsumsi sebagai prasyarat validasi pembayaran pesanan (Dependency Wajib).

2.3 Diagram FlowChart (Alur Transaksi & Integrasi)

Berikut adalah diagram alur proses bisnis utama, mulai dari autentikasi, pemesanan, validasi pembayaran ke sistem mitra, hingga penugasan pengemudi.



Penjelasan Alur:

1. Autentikasi: Klien melakukan login melalui *User Service* untuk mendapatkan JWT Token.
2. Pemesanan: Klien memilih menu dari *Restaurant Service* dan membuat pesanan ke *Order Service*.
3. Validasi Internal: *Order Service* memastikan stok menu tersedia.
4. Integrasi Pembayaran: *Order Service* meminta *Payment Service* untuk memproses pembayaran. Pada tahap ini, sistem melakukan *request* ke API GraphQL milik DOSWallet (Mitra) untuk memotong saldo pengguna.
5. Penugasan Driver: Jika pembayaran sukses (PAID), *Order Service* secara otomatis menghubungi *Driver Service* untuk mencari pengemudi yang tersedia.
6. Penyelesaian: Status pesanan diperbarui, dan klien menerima konfirmasi pesanan beserta data pengemudi.

BAB 3 PEMBAGIAN PERAN

Struktur ini mengikuti arsitektur sistem yang terdiri dari 5 layanan mikro, dengan distribusi tanggung jawab sebagai berikut:

Anggota 1 (102022330145 - Muhammad Rayhan Ramadhan) — System Architect & Integrator

Service: Core Commerce Services (Mencakup: *User Service*, *Restaurant Service*, & *Order Service*) Fokus: Arsitektur utama sistem, manajemen identitas (Auth), katalog produk dinamis, dan orkestrasi transaksi pemesanan (Order).

Ruang Kerja Teknis:

1. Authentication & Identity (User Service):
 - a. Implementasi login/register menghasilkan Access Token JWT dengan klaim: sub (user_id), email, role (Customer/Admin/Driver).
 - b. Manajemen profil pengguna dan alamat pengiriman sebagai *single source of truth*.
 - c. Keamanan password menggunakan *hashing* bcrypt.
2. Catalog Management (Restaurant Service):
 - a. CRUD Data Restoran dan Menu Makanan.
 - b. Manajemen stok dan ketersediaan menu secara *real-time*.
3. Transaction Orchestration (Order Service):
 - a. Menangani siklus hidup pesanan (*Created -> Paid -> Finding Driver -> Completed*).
 - b. Mengonsumsi data dari *User Service* (validasi token) dan *Restaurant Service* (validasi menu/harga) sebelum pesanan dibuat.
 - c. Integrasi Eksternal (Wajib): Bertanggung jawab memanggil endpoint GraphQL DOSWallet (Mitra) untuk memvalidasi pembayaran secara *server-to-server*.

GraphQL Provider:

1. Query Minimal:
 - a. me: Profil user saat ini.
 - b. restaurants: Daftar restoran dan menunya.
 - c. myOrders: Riwayat pesanan user.
2. Mutation Minimal:
 - a. login, register.

- b. `createOrder(input)`: Mutasi kompleks yang memicu validasi stok dan pembayaran eksternal.

Database: MySQL (`user_db`, `restaurant_db`, `order_db` - *Database per Service pattern*). Tech Stack: Node.js (Express) + Apollo Server + TypeORM/Prisma + Docker.

Anggota 2 (102022300082 - Darvesh Gladwin Musyaffa) — Financial Transaction Manager

Service: Payment Service (Internal Ledger & Gateway Wrapper) Fokus: Pencatatan riwayat transaksi keuangan internal dan penanganan respon pembayaran.

Ruang Kerja Teknis:

1. Payment Recording:
 - a. Mencatat setiap transaksi masuk yang berhasil diverifikasi dari sistem eksternal.
 - b. Menyimpan status pembayaran (PENDING, PAID, FAILED) untuk kebutuhan audit internal.
2. Gateway Interface:
 - a. Menyediakan abstraksi/interface untuk komunikasi dengan berbagai metode pembayaran (dalam hal ini disimulasikan koneksi ke E-Wallet).

GraphQL Provider:

1. Query Minimal:
 - a. `paymentHistory(orderId: ID!)`: Melihat status pembayaran spesifik.
2. Mutation Minimal:
 - a. `recordPayment`: Mencatat konfirmasi pembayaran sukses.

Database: MySQL (`payment_db`). Tech Stack: Node.js (Express) + Apollo Server + TypeORM + Docker.

Anggota 3 (102022300416 - Jehezkiel Agna Saputra) — Fleet Operations Manager

Service: Driver Service Fokus: Manajemen armada pengemudi, status ketersediaan, dan alokasi pengiriman.

Ruang Kerja Teknis:

1. Driver Management:
 - a. Registrasi dan manajemen profil pengemudi.
 - b. Fitur *toggle* status driver (ONLINE / OFFLINE).
2. Order Assignment:
 - a. Logika penerimaan pesanan dari *Order Service* (ketika status PAID).
 - b. Mengupdate lokasi pengemudi (simulasi) dan status pengantaran.

GraphQL Provider:

1. Query Minimal:

- a. availableDrivers: Mengambil list driver yang sedang online.
- b. driverProfile: Profil detail mitra pengemudi.

2. Mutation Minimal:

- a. updateDriverStatus(status): Ubah status ketersediaan.
- b. acceptOrder(orderId): Driver menerima pesanan masuk.

Database: MySQL (driver_db). Tech Stack: Node.js (Express) + Apollo Server + TypeORM
+ Docker.

BAB 4 MEKANISME JWT & MODEL KEAMANAN

4.1 Prinsip “Independent Auth”

Sistem ini menerapkan prinsip keamanan terdesentralisasi namun terpusat pada penerbitan identitas. Desain keamanannya mengikuti aturan berikut:

1. Single Source of Credentials: Hanya User Service yang memiliki akses ke tabel users dan menyimpan *hash* password. Layanan lain seperti *Order Service*, *Restaurant Service*, *Driver Service*, dan *Payment Service* tidak memiliki tabel user dan tidak pernah menyimpan kredensial pengguna.
2. Stateless Authentication: Klien (Frontend) menyimpan JWT (*JSON Web Token*) yang diterbitkan saat login. Setiap permintaan ke layanan bisnis (*misal: membuat pesanan di Order Service*) wajib menyertakan token ini dalam *Header Authorization*.
3. On-Demand Identity Data: Jika layanan bisnis membutuhkan data detail pengguna yang tidak ada dalam *payload* token (misalnya alamat pengiriman lengkap untuk *Driver Service*), layanan tersebut akan melakukan *query GraphQL* ke *User Service* menggunakan ID yang didapat dari token (sub).

4.2 Opsi Validasi Token: Stateless Verification (Terpilih)

Untuk memaksimalkan performa dan mengurangi latensi jaringan antar-layanan, sistem ini memilih Opsi A — Stateless Verification dengan algoritma tanda tangan HMAC SHA-256 (HS256) atau RSA Signature (RS256).

1. Mekanisme Kerja:
 - a. Penerbitan: *User Service* menerbitkan token yang ditandatangani dengan *Private Key* (atau *Secret Key*).
 - b. Verifikasi: Layanan lain (*Order, Restaurant, Payment*) memiliki salinan *Public Key* (atau *Shared Secret*) yang disuntikkan melalui *Environment Variables*.
 - c. Proses: Saat *request* masuk, layanan penerima memvalidasi tanda tangan (*signature*) dan masa berlaku (*expiration*) token secara lokal tanpa perlu memanggil *User Service*.
2. Keuntungan: Mengurangi beban trafik ke *User Service* dan mempercepat waktu respons API karena tidak ada *round-trip* HTTP tambahan untuk validasi.

4.3 Kebijakan Token & Penanganan Error

Sistem menerapkan kebijakan keamanan minimal sebagai berikut:

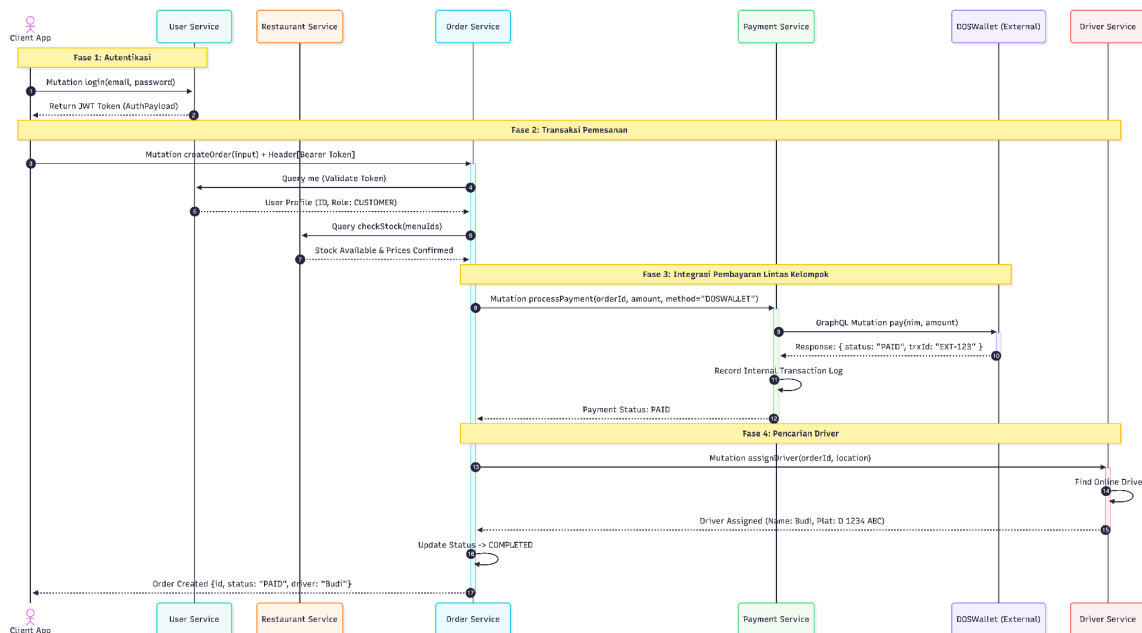
1. Masa Berlaku Token (TTL):
 - a. *Access Token* bersifat *short-lived* (berlaku selama 60 menit) untuk meminimalisir risiko penyalahgunaan jika token dicuri.

2. Klaim Token Minimal (Payload): Token memuat informasi esensial agar layanan lain dapat mengambil keputusan otorisasi dasar:
 - a. sub: User ID (Primary Key).
 - b. email: Identitas pengguna.
 - c. role: Peran pengguna (CUSTOMER, DRIVER, atau ADMIN) untuk *Role-Based Access Control* (RBAC).
3. Standar Penanganan Error HTTP:
 - a. 401 Unauthorized: Dikembalikan jika token tidak ada, format salah (*Malformed*), tanda tangan tidak valid, atau sudah kedaluwarsa (*Expired*).
 - b. 403 Forbidden: Dikembalikan jika token valid, tetapi role pengguna tidak memiliki izin untuk mengakses sumber daya tersebut (misal: CUSTOMER mencoba mengakses API khusus ADMIN di *Restaurant Service*).

BAB 5 ALUR INTEGRASI

5.1 Sequence Diagram (Integrasi Lintas Kelompok)

Diagram berikut mengilustrasikan orkestrasi layanan saat terjadi transaksi pemesanan. Fokus utama adalah pada Order Service yang bertindak sebagai koordinator, memanggil Restaurant Service untuk validasi produk, dan Payment Service yang terhubung ke DOSWallet (Sistem Eksternal) untuk pemotongan saldo.



5.2 Aturan Bisnis Minimal Pemesanan (Order Rules)

Agar transaksi pemesanan dapat diproses dan diselesaikan (*Settlement*), sistem menerapkan aturan validasi ketat yang melibatkan kolaborasi antar-layanan (*Service Orchestration*) sebagai berikut:

- Validasi Identitas & Peran (User Service)
 - Transaksi hanya dapat diinisiasi jika Token JWT valid (tidak *expired* dan *signature* sesuai).
 - Pengguna harus memiliki role CUSTOMER. Role DRIVER atau ADMIN tidak diizinkan membuat pesanan makanan.
- Validasi Ketersediaan Produk (Restaurant Service)
 - Menu yang dipesan harus terdaftar pada restoran yang aktif (*Open*).
 - Stok menu harus mencukupi jumlah yang diminta (*Quantity Check*). Jika stok habis, pesanan ditolak seketika.
- Status Pembayaran Eksternal (Integration Rule)
 - Pemesanan WAJIB lunas di muka (*Prepaid*).

- b. Order Service tidak akan meneruskan pesanan ke dapur restoran atau driver sebelum menerima konfirmasi status PAID dari API DOSWallet (Kelompok Mitra). Jika saldo di DOSWallet tidak mencukupi, transaksi dibatalkan.
- 4. Ketersediaan Armada (Driver Service)
 - a. Pesanan hanya dapat diselesaikan jika terdapat setidaknya satu pengemudi dengan status ONLINE di wilayah layanan.

BAB 6 KONTRAK GRAPHQL

Sistem ini menerapkan pendekatan *Schema-First Development*. Setiap layanan mendefinisikan kontrak tipe data (*Type Definitions*) secara eksplisit untuk menjamin konsistensi komunikasi data antar 5 layanan mikro.

6.1 Auth & User Service Schema

Layanan ini menangani otorisasi dan profil pengguna.

```
type User {
```

```
  id: ID!
```

```
  name: String!
```

```
  email: String!
```

```
  phone: String
```

```
  role: String! # CUSTOMER, DRIVER, ADMIN
```

```
}
```

```
type AuthPayload {
```

```
  token: String!
```

```
  user: User!
```

```
}
```

```
type Query {
```

```
  me: User # Mengambil profil user berdasarkan token Bearer
```

```
  userById(id: ID!): User # Endpoint publik untuk integrasi antar service
```

```
}
```

```
type Mutation {
```

```
  register(name: String!, email: String!, password: String!, role: String!): AuthPayload
```

```
  login(email: String!, password: String!): AuthPayload
```

```
}
```

6.2 Restaurant Service Schema

Layanan katalog yang menyediakan struktur data hierarkis (Restoran -> Menu).

```
type Menu {
```

```
  id: ID!
```

```
  name: String!
```

```
    description: String
    price: Float!
    isAvailable: Boolean!
}
```

```
type Restaurant {
  id: ID!
  name: String!
  address: String!
  menus: [Menu]!
}
```

```
type Query {
  restaurants: [Restaurant]!
  restaurant(id: ID!): Restaurant
  menu(restaurantId: ID!): [Menu]!
}
```

```
type Mutation {
  createRestaurant(name: String!, address: String!): Restaurant
  addMenu(restaurantId: ID!, input: MenuInput!): Menu
}
```

6.3 Order Service Schema (Core Transaction)

Layanan pusat yang mengelola siklus hidup pesanan.

```
type OrderItem {
  menuId: String!
  quantity: Int!
  priceAtPurchase: Float!
}
```

```
type Order {
  id: ID!
  userId: String!
```

```

restaurantId: String!
driverId: String # Diisi setelah Driver Service assign driver
totalAmount: Float!
status: String! # PENDING, PAID, FINDING_DRIVER, ON_DELIVERY, COMPLETED
}

```

```

input CreateOrderInput {
  restaurantId: String!
  items: [OrderItemInput]!
  paymentMethod: String! # Default: "DOSWALLET"
}

```

```

type Mutation {
  createOrder(input: CreateOrderInput!): Order
  cancelOrder(id: ID!): Boolean
}

```

```

type Query {
  myOrders: [Order]!
  order(id: ID!): Order
}

```

6.4 Payment Service Schema (Internal Ledger)

Layanan yang mencatat status transaksi dan berinteraksi dengan gateway eksternal.

```

type PaymentLog {
  id: ID!
  orderId: String!
  amount: Float!
  status: String! # SUCCESS, FAILED
  transactionDate: String!
  externalReferenceId: String # ID dari DOSWallet
}

```

```

type Query {

```

```
    paymentHistory(orderId: ID!): [PaymentLog]
  }
```

```
type Mutation {
  # Dipanggil oleh Order Service saat checkout
  processPayment(orderId: ID!, amount: Float!, userId: String!): PaymentLog
}
```

6.5 Driver Service Schema (Fleet Management)

Layanan manajemen armada pengiriman.

```
type Driver {
  id: ID!
  userId: String! # Link ke User Service
  licensePlate: String!
  isOnline: Boolean!
  currentLocation: String
}
```

```
type Query {
  availableDrivers: [Driver]!
  driverProfile(id: ID!): Driver
}
```

```
type Mutation {
  toggleStatus(isOnline: Boolean!): Driver
  assignDriver(orderId: ID!, location: String!): Driver # Dipanggil Order Service
  completeDelivery(orderId: ID!): Boolean
}
```

BAB 7 STRATEGI DEPLOYMENT DOCKER & TOPOLOGI JARINGAN

Untuk memenuhi aspek penilaian "Docker Deployment", seluruh arsitektur dijalankan menggunakan Docker Compose dengan prinsip isolasi total.

7.1 Topologi Container

Sistem terdiri dari 11 container yang berjalan secara orkestratif:

1. Service Containers (Application Layer):
 - a. user-service (Node.js/Express + Apollo)
 - b. restaurant-service (Node.js/Express + Apollo)
 - c. order-service (Node.js/Express + Apollo)
 - d. payment-service (Node.js/Express + Apollo)
 - e. driver-service (Node.js/Express + Apollo)
 - f. api-gateway (Opsional: Nginx/Apollo Gateway sebagai entry point tunggal)
2. Database Containers (Data Layer):
 - a. user_db, restaurant_db, order_db, payment_db, driver_db (MySQL 8.0)
 - b. *Catatan:* Menggunakan pattern Database-per-Service untuk mencegah *tight coupling*.

7.2 Konfigurasi Jaringan (Networking)

Semua container terhubung dalam satu *bridge network* internal bernama iae-network. Komunikasi antar-service menggunakan *Service Discovery* (memanggil nama container, bukan IP), contoh: `http://user-service:4001/graphql`.

BAB 8 RENCANA INTEGRASI LINTAS KELOMPOK

Sistem ini dirancang untuk interoperabilitas dengan ekosistem luar melalui pertukaran data via GraphQL API.

8.1 Sebagai Consumer (Pengguna Layanan Eksternal)

Skenario: Pembayaran Pesanan Makanan menggunakan E-Wallet Mitra. Layanan Payment Service kami bertindak sebagai *client* yang mengonsumsi API dari kelompok DOSWallet.

1. Target Endpoint: `http://doswallet-service-url/graphql`

Query/Mutation yang Dikonsumsi:

```
mutation PayOrder($nim: String!, $amount: Float!) {  
  pay(nim: $nim, amount: $amount) {  
    status # Mengharapkan return "SUCCESS"  
    trxId  
    balanceRemaining  
  }  
}
```

2. Dampak Bisnis: Jika respon status bukan "SUCCESS", maka Order Service akan membatalkan pesanan secara otomatis.

8.2 Sebagai Provider (Penyedia Data bagi Kelompok Lain)

Skenario: Validasi Profil Mahasiswa. Sistem kami membuka akses data profil pengguna agar dapat dimanfaatkan oleh kelompok lain (misalnya sistem *Academic Dashboard* yang ingin menampilkan riwayat jajan mahasiswa).

1. Endpoint Publik: `http://food-delivery-gateway/graphql`

Query yang Disediakan:

```
query GetUserProfile($id: ID!) {  
  userById(id: $id) {  
    name  
    email  
    role  
  }  
}
```

BAB 9 IMPLEMENTASI DAN HASIL INTEGRASI SISTEM

9.1 Realisasi Infrastruktur Microservices

Implementasi infrastruktur sistem Food Delivery ini sepenuhnya dibangun di atas teknologi kontainerisasi menggunakan Docker, yang memungkinkan isolasi lingkungan kerja antara layanan *merchant* (Food Delivery System/FDS) dan layanan mitra pembayaran (*DosWallet*). Berbeda dengan rancangan monolitik tradisional, realisasi ini memecah sistem menjadi unit-unit independen yang di orkestrasi melalui docker-compose.

9.1.1 Topologi Kontainer dan Orkestrasi Layanan

Pada tahap implementasi akhir, sistem berhasil menjalankan total 14 kontainer aktif secara bersamaan. Lingkungan Food Delivery System (FDS) menjalankan 7 layanan utama: lima *microservices* backend berbasis Python (User, Restaurant, Order, Payment, Driver), satu API Gateway berbasis Node.js sebagai pintu gerbang lalu lintas data, dan satu Frontend berbasis React. Disisi lain, ekosistem mitra *DosWallet* berjalan secara paralel dengan 3 layanan backend dan 1 frontend terpisah.

Setiap layanan backend memiliki basis data MySQL terdedikasi (*Database per Service pattern*), menjamin *loose coupling* di mana kegagalan pada satu basis data tidak akan mematikan seluruh sistem. Bukti keberhasilan orkestrasi ini terlihat pada status kontainer yang seluruhnya dalam kondisi Up (Running) dan Healthy, menandakan bahwa manajemen dependensi antar layanan telah terkonfigurasi dengan benar.

rayyyhann@192 ~ % docker ps						
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
92832627d5c1	doswallet-web-dw-transaction-service	"python app.py"	21 minutes ago	Up 21 minutes	0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp	doswallet-t
e9eb7a645b7c	doswallet-web-dw-wallet-service	"python app.py"	21 minutes ago	Up 21 minutes	0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp	doswallet-w
287807facc6e	doswallet-web-dw-user-service	"python app.py"	21 minutes ago	Up 21 minutes	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp	doswallet-u
448286b16bb9	doswallet-web-dw-notification-service	"python app.py"	21 minutes ago	Up 21 minutes	0.0.0.0:5004->5004/tcp, [::]:5004->5004/tcp	doswallet-n
256b45a97da7	mysql:8.0	"docker-entrypoint.s..."	21 minutes ago	Up 21 minutes (healthy)	0.0.0.0:3310->3306/tcp, [::]:3310->3306/tcp	doswallet-n
e7ce463f134a	doswallet-web-frontend-web	"docker-entrypoint.s..."	21 minutes ago	Up 21 minutes	0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp	doswallet-f
2471b6a7772f	uasiae-fooodeliverysystem-withintegration-order-service	"uvicorn app.main:ap..."	4 days ago	Up About an hour	0.0.0.0:4003->8000/tcp, [::]:4003->8000/tcp	uasiae-food
cf99068c127d	uasiae-fooodeliverysystem-withintegration-payment-service	"uvicorn app.main:ap..."	4 days ago	Up About an hour	0.0.0.0:4004->8000/tcp, [::]:4004->8000/tcp	uasiae-food
5e8a3ecf4c68	uasiae-fooodeliverysystem-withintegration-restaurant-service	"uvicorn app.main:ap..."	4 days ago	Up About an hour	0.0.0.0:4002->8000/tcp, [::]:4002->8000/tcp	uasiae-food
5b0b7f9be6d6	uasiae-fooodeliverysystem-withintegration-frontend	"/docker-entrypoint..."	9 days ago	Up About an hour	0.0.0.0:80->80/tcp, [::]:80->80/tcp	uasiae-food
f4dc7d62efb3	uasiae-fooodeliverysystem-withintegration-api-gateway	"docker-entrypoint.s..."	9 days ago	Up About an hour	0.0.0.0:4000->3000/tcp, [::]:4000->3000/tcp	uasiae-food
cc4722d34e4f	uasiae-fooodeliverysystem-withintegration-driver-service	"uvicorn app.main:ap..."	9 days ago	Up About an hour	0.0.0.0:4005->8000/tcp, [::]:4005->8000/tcp	uasiae-food
de3843a9935a	uasiae-fooodeliverysystem-withintegration-user-service	"uvicorn app.main:ap..."	9 days ago	Up About an hour	0.0.0.0:4001->8000/tcp, [::]:4001->8000/tcp	uasiae-food
f498f174bd3a	mysql:8.0	"docker-entrypoint.s..."	9 days ago	Up About an hour	3306/tcp, 33060/tcp	uasiae-food
36c9dbed26ab	mysql:8.0	"docker-entrypoint.s..."	9 days ago	Up About an hour	3306/tcp, 33060/tcp	uasiae-food
855719ea241a	mysql:8.0	"docker-entrypoint.s..."	9 days ago	Up About an hour	3306/tcp, 33060/tcp	uasiae-food
d648ae56f9d	mysql:8.0	"docker-entrypoint.s..."	9 days ago	Up About an hour	3306/tcp, 33060/tcp	uasiae-food

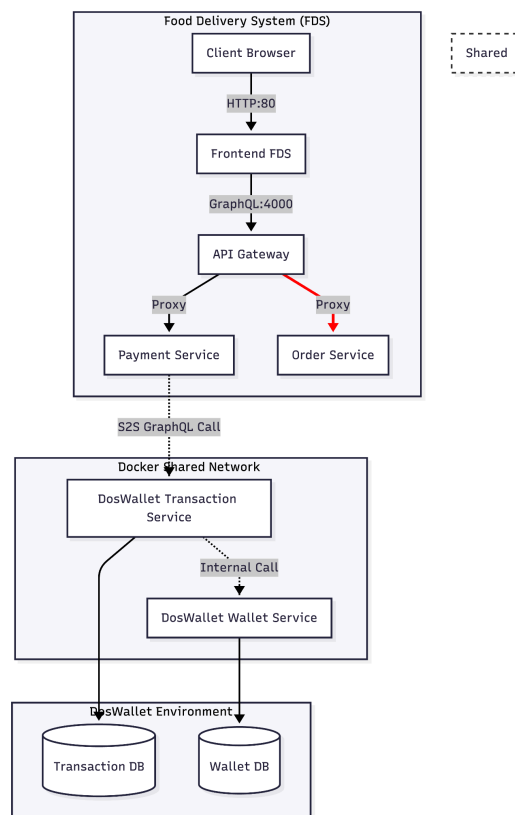
9.1.2 Konfigurasi Jaringan Terintegrasi (Shared Network)

Tantangan utama dalam integrasi dua sistem terpisah (FDS dan DosWallet) adalah komunikasi antar-kontainer yang berada dalam *project namespace* yang berbeda. Untuk mengatasi hal ini, implementasi jaringan tidak menggunakan mode bridge standar yang

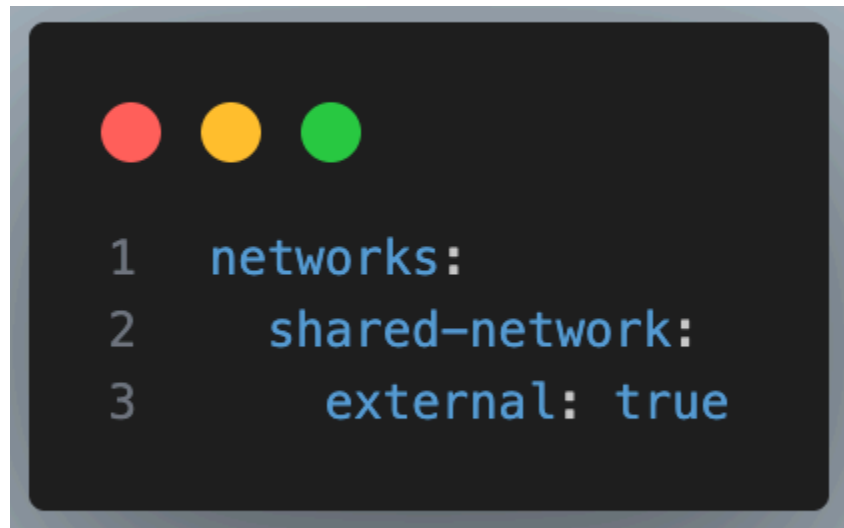
terisolasi, melainkan menggunakan External Named Network yang diberi nama shared-network.

Konfigurasi ini memungkinkan terbentuknya jembatan komunikasi langsung (*Direct Service-to-Service Communication*) antara payment-service milik FDS dan dw-transaction-service milik DosWallet tanpa harus melalui internet publik. Dalam arsitektur jaringan ini, setiap kontainer dapat saling mengenali menggunakan *DNS Resolution* internal Docker berdasarkan nama layanan (misalnya: memanggil `http://dw-transaction-service:5003`).

Berikut adalah visualisasi topologi jaringan yang telah diimplementasikan:



Realisasi konfigurasi jaringan ini dibuktikan dengan deklarasi networks pada file docker-compose.yml di kedua proyek, yang secara eksplisit mendefinisikan penggunaan jaringan eksternal sebagai berikut:



Dengan konfigurasi ini, latensi komunikasi antar layanan dapat diminimalisir dan keamanan transaksi lebih terjamin karena pertukaran data sensitif (seperti *Secret Key* dan *Token*) terjadi di dalam jaringan internal Docker yang tertutup dari akses luar yang tidak terotorisasi.

9.2 Implementasi Backend GraphQL (Server-Side)

Transformasi fundamental dalam pengembangan sistem ini adalah migrasi dari arsitektur RESTful API konvensional menjadi GraphQL menggunakan pustaka Strawberry (Python). Pendekatan ini dipilih untuk mengatasi masalah *over-fetching* dan *under-fetching* data yang sering terjadi pada komunikasi mikroservis, serta untuk menyediakan satu *endpoint* terpadu bagi klien.

9.2.1 Definisi Skema dan Resolver (Strawberry GraphQL)

Setiap layanan backend (User, Restaurant, Order, Payment, Driver) kini mengekspos *endpoint* tunggal `/graphql`. Struktur data tidak lagi didefinisikan sebagai respon JSON statis, melainkan sebagai Type Definitions yang mencerminkan model basis data SQLAlchemy. Implementasi ini memungkinkan *frontend* untuk meminta data secara spesifik. Sebagai contoh, pada layanan Restoran, klien dapat memilih untuk hanya mengambil nama dan harga menu tanpa harus memuat informasi detail lainnya yang tidak diperlukan.

```

1  @strawberry.type
2  class RestaurantType:
3      id: int
4      name: str
5      address: str
6      isOpen: bool
7      category: Optional[str]
8      imageUrl: Optional[str]
9
10 @strawberry.field
11 def menus(self) -> List[MenuItemType]:
12     db = SessionLocal()
13     menus = db.query(MenuItem).filter(MenuItem.restaurant_id == self.id).all()
14     db.close()
15     return [
16         MenuItemType(
17             id=m.id,
18             name=m.name,
19             description=m.description,
20             price=float(m.price),
21             stock=m.stock,
22             category=m.category,
23             imageUrl=m.image_url,
24             isAvailable=bool(m.is_available)
25         ) for m in menus
26     ]

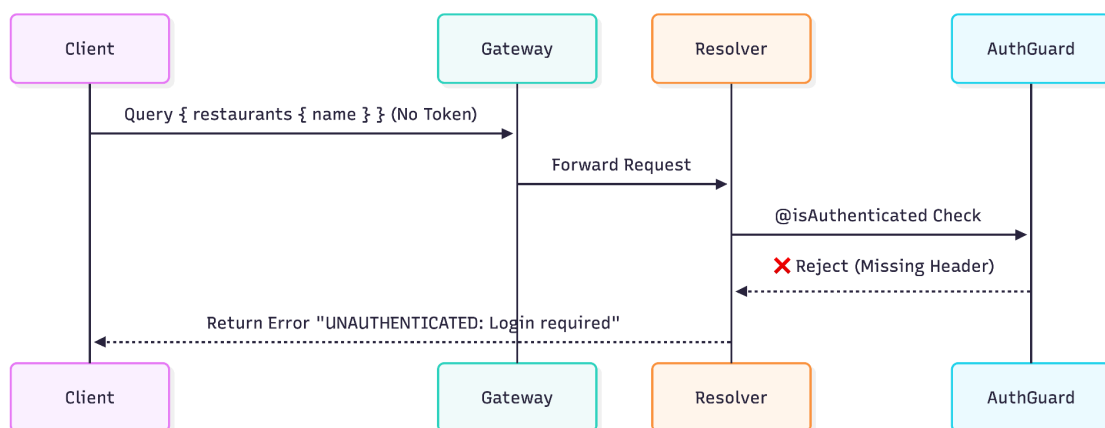
```

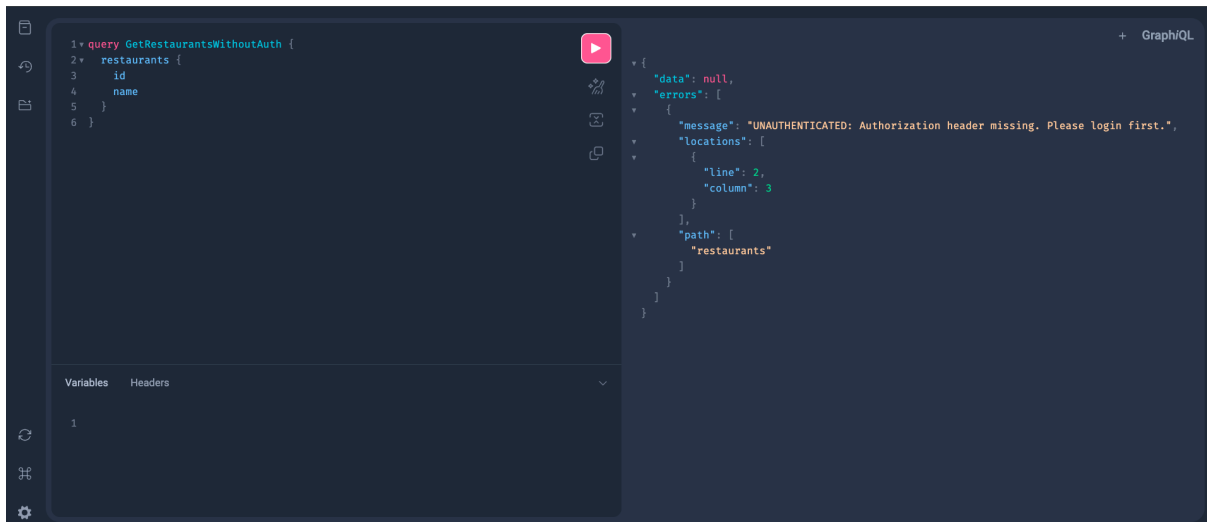
9.2.2 Mekanisme Keamanan Ketat (*Strict Authentication*)

Aspek keamanan menjadi prioritas utama dalam implementasi GraphQL ini. Sistem menerapkan kebijakan Strict Auth menggunakan *Permission Classes* pada level *Resolver*. Berbeda dengan desain awal yang mungkin mengizinkan akses publik untuk data tertentu, implementasi final mewajibkan penyertaan JWT (JSON Web Token) pada Header Authorization untuk seluruh operasi data sensitif (Query Restoran, Create Order, Payment).

Mekanisme ini bekerja dengan cara mengintersepsi setiap *request* yang masuk. Jika token tidak ditemukan atau tidak valid, sistem secara otomatis menolak permintaan dengan kode error standar UNAUTHENTICATED, sebelum logika bisnis dijalankan.

Berikut adalah diagram alur validasi keamanan yang diterapkan:





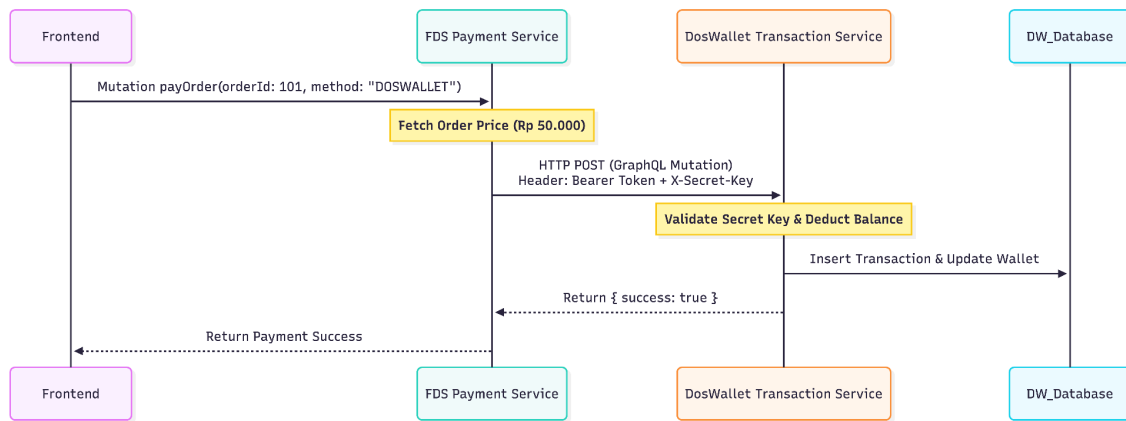
9.2.3 Logika Pembayaran Antar-Layanan (*Server-to-Server GraphQL Call*)

Puncak dari integrasi sistem ini terletak pada layanan pembayaran. Fitur pembayaran menggunakan *DosWallet* tidak dilakukan melalui *redirect* URL sederhana, melainkan melalui komunikasi langsung antar-server (*Server-to-Server Communication*).

Ketika mutasi `payOrder` dipanggil oleh klien, Payment Service (FDS) bertindak sebagai *klien GraphQL* yang menghubungi Transaction Service (*DosWallet*). Proses ini melibatkan langkah-langkah teknis berikut:

1. Konstruksi Payload: Payment Service menyusun *query mutation* GraphQL secara dinamis berisi `amount` dan `order_id`.
2. Penandatanganan Keamanan: Permintaan dilengkapi dengan Header `X-Secret-Key` yang telah disepakati kedua sistem untuk memvalidasi bahwa permintaan berasal dari *merchant* resmi.
3. Eksekusi Transaksi: *DosWallet* memproses pemotongan saldo dan mengembalikan status sukses, yang kemudian memicu perubahan status pesanan di FDS menjadi **PAID**.

Visualisasi logika komunikasi antar-server tersebut digambarkan sebagai berikut:



```

1+ mutation CreateOrder {
2   createOrder(
3     restaurantId: 1
4     addressId: 1
5     items: [
6       { menuItemId: 1, quantity: 2 }
7     ]
8   ) {
9     id
10    userId
11    restaurantId
12    status
13    totalPrice
14    items {
15      id
16      menuItemName
17      quantity
18      price
19    }
20  }
}

```

Variables Headers

```

1+ {
2   "Content-Type": "application/json",
3   "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjdXN0b211
4 }

```

GraphQL Response:

```

{
  "data": {
    "createOrder": {
      "id": 25,
      "userId": 5,
      "restaurantId": 1,
      "status": "PENDING_PAYMENT",
      "totalPrice": 50000,
      "items": [
        {
          "id": 37,
          "menuItemName": "Sate Padang (Daging)",
          "quantity": 2,
          "price": 25000
        }
      ]
    }
  }
}

```

```

1+ mutation PayOrderWithDosWallet {
2+   payOrder(orderId: 25, method: "DOSWALLET") {
3     success
4     message
5+    payment {
6      id
7      orderId
8      userId
9      amount
10     status
11     paymentMethod
12     createdAt
13   }
14 }
15 }

```

Variables Headers

```

1+ {
2   "Content-Type": "application/json",
3   "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjdXN0b211
4 }

```

GraphQL Response:

```

{
  "data": {
    "payOrder": {
      "success": true,
      "message": "Payment successful",
      "payment": {
        "id": 3,
        "orderId": 25,
        "userId": 5,
        "amount": 50000,
        "status": "SUCCESS",
        "paymentMethod": "DOSWALLET",
        "createdAt": "2026-01-09 01:41:31"
      }
    }
  }
}

```

```

rayyyhann@192 DOSWALLET-WEB % curl -s -X POST http://localhost:5003/graphql \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-d '{"query": "query { myTransactions(limit: 5) { transaction_id user_id amount type description status } }"}' \
| python3 -m json.tool
{
  "data": {
    "myTransactions": [
      {
        "transaction_id": 14,
        "user_id": 1,
        "amount": 50000.0,
        "type": "withdraw",
        "description": "Payment for Order 25",
        "status": "completed"
      },
      {
        "transaction_id": 13,
        "user_id": 1,
        "amount": 50000.0,
        "type": "withdraw",
        "description": "Payment for Order 23",
        "status": "completed"
      },
      {
        "transaction_id": 12,
        "user_id": 1,
        "amount": 50000.0,
        "type": "withdraw",
        "description": "Payment for Order 22",
        "status": "completed"
      },
      {
        "transaction_id": 11,
        "user_id": 1,
        "amount": 25000.0,
        "type": "withdraw",
        "description": "Payment for Order 19",
        "status": "completed"
      },
      {
        "transaction_id": 9,
        "user_id": 1,
        "amount": 126000.0,
        "type": "withdraw",
        "description": "Payment for Order 18",
        "status": "completed"
      }
    ]
  }
}

```

9.3 Integrasi Frontend (Client-Side Implementation)

Implementasi sisi klien (*Client-Side*) dibangun menggunakan kerangka kerja React (Vite) yang telah dimodifikasi untuk mendukung komunikasi berbasis GraphQL. Fokus utama pengembangan antarmuka ini adalah menciptakan pengalaman pengguna yang mulus (*seamless*) saat berpindah antara ekosistem *Food Delivery System* (FDS) dan *DosWallet*, tanpa mengorbankan keamanan data pengguna.

9.3.1 Konfigurasi Klien GraphQL dan Manajemen State

Berbeda dengan implementasi REST API sebelumnya yang mengandalkan banyak *endpoint* terpisah, integrasi ini memusatkan seluruh pengambilan data melalui satu *layer* layanan API (*api.ts*). Modul ini bertanggung jawab untuk menyusun *query* dan *mutation* GraphQL, serta menyisipkan JWT Authorization Token secara otomatis pada setiap permintaan ke *Gateway*.

Penerapan pola ini memungkinkan *frontend* untuk menangani *error handling* secara terpusat. Jika *Gateway* mengembalikan respon UNAUTHENTICATED, aplikasi secara otomatis akan mengarahkan pengguna kembali ke halaman Login, menjaga integritas sesi pengguna.



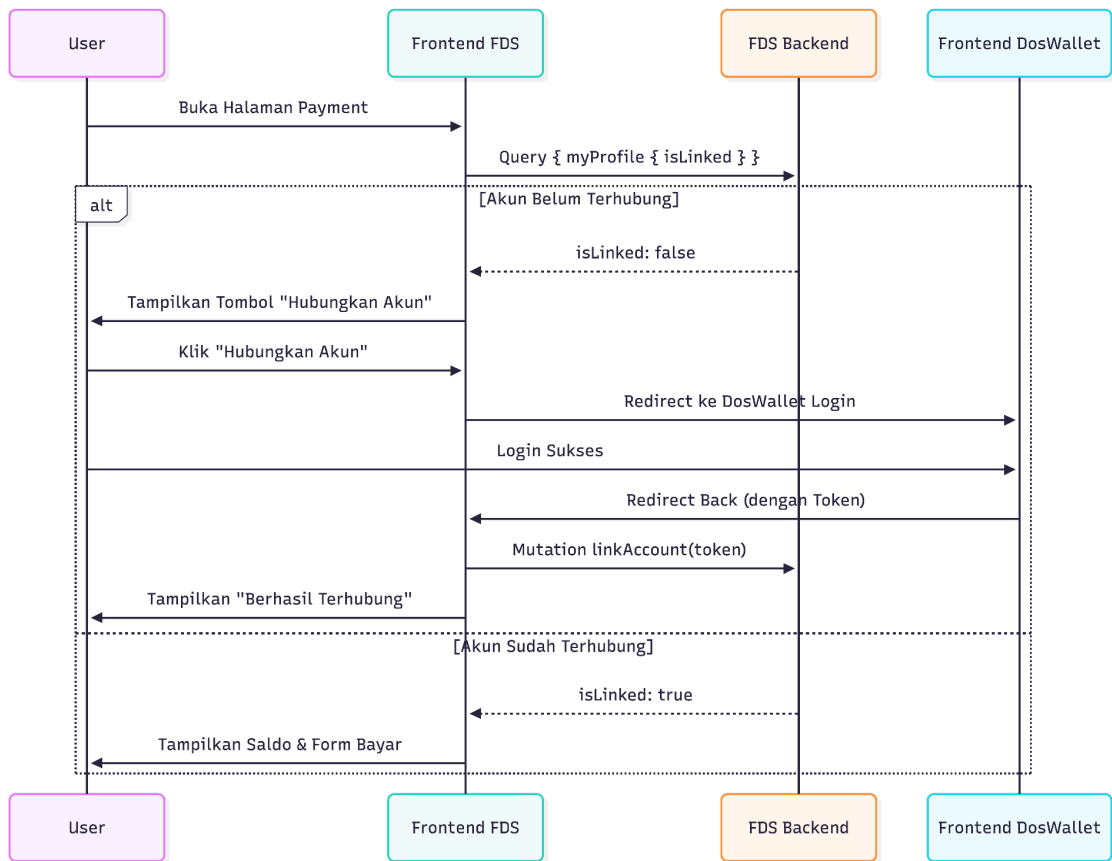
```
1  const api = axios.create({
2    baseURL: API_BASE_URL,
3    timeout: 15000, // allow slower local services before timing out
4    headers: {
5      'Content-Type': 'application/json',
6    },
7  });
```

9.3.2 Alur Penghubungan Akun Cerdas (*Smart Account Linking*)

Fitur unggulan pada antarmuka pengguna adalah mekanisme Smart Account Linking. Sistem secara cerdas mendeteksi status integrasi pengguna saat halaman Pembayaran dimuat. Logika UI bekerja sebagai berikut:

1. Deteksi Status: Aplikasi melakukan *query* ke backend untuk memeriksa status *is_linked*.
2. Tampilan Kondisional:
 - i. Jika Belum Terhubung: Tombol pembayaran dinonaktifkan dan diganti dengan tombol aksi "*Hubungkan ke DosWallet*".
 - ii. Jika Sudah Terhubung: Saldo ditampilkan dan formulir pembayaran diaktifkan.
3. Redireksi Aman: Saat tombol hubungkan ditekan, aplikasi melakukan *redirect* pengguna ke domain *DosWallet* dengan membawa parameter *callback_url*. Setelah pengguna sukses *login* di *DosWallet*, mereka dikembalikan ke FDS secara otomatis.

Berikut adalah diagram alur interaksi antarmuka (*UI Flow*) untuk proses tersebut:



Food Delivery

Browse
My Orders
Profile
glet
Logout

Subtotal
Tax (10%)
Delivery Fee
Total

Rp 52.000
Rp 5.200
Rp 10.000
Rp 52.000

Payment Method

☐ E-Wallet

☒ DosWallet

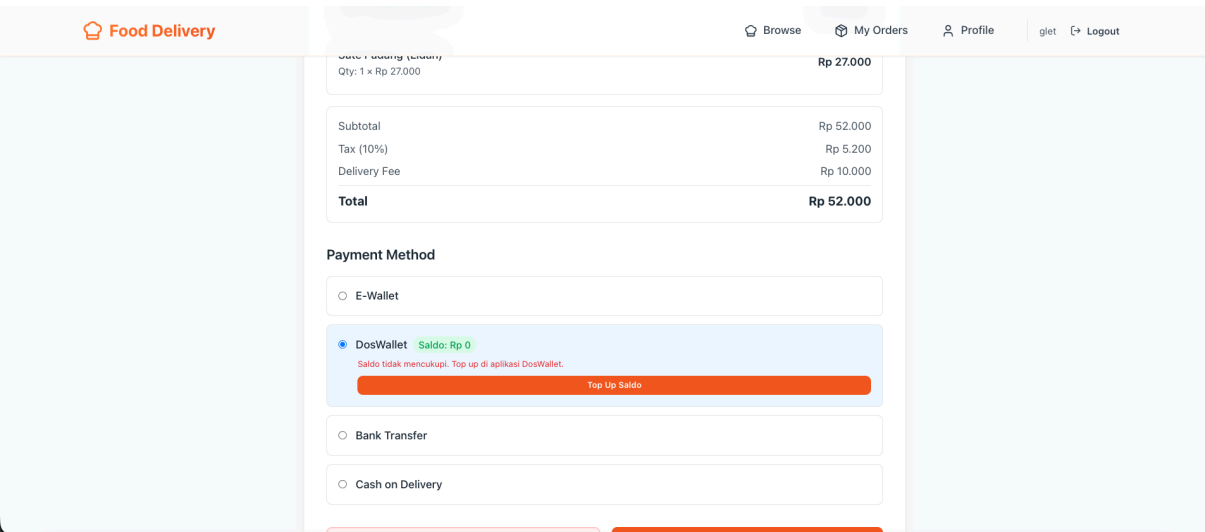
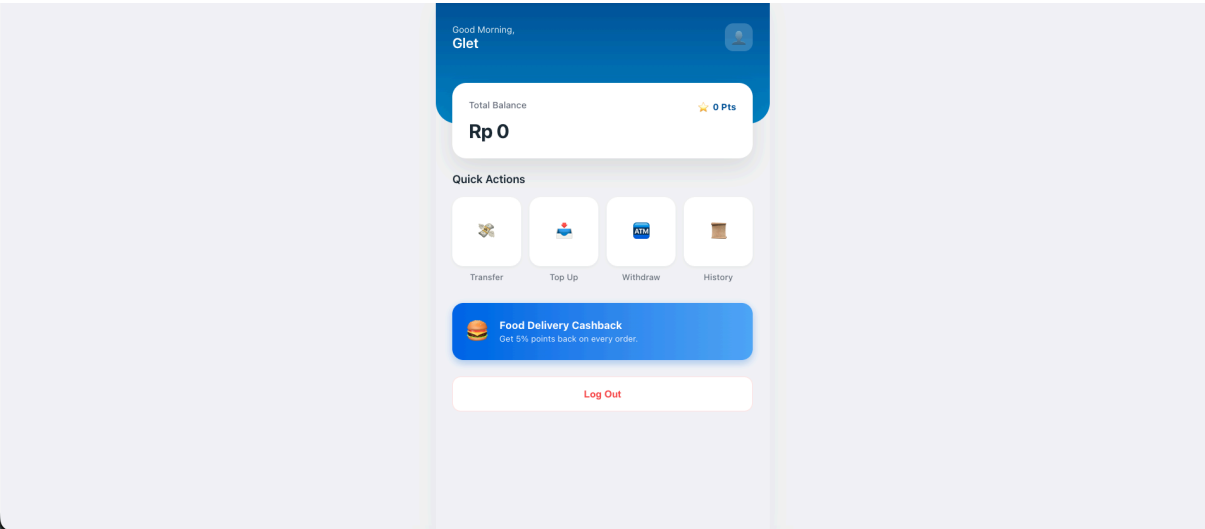
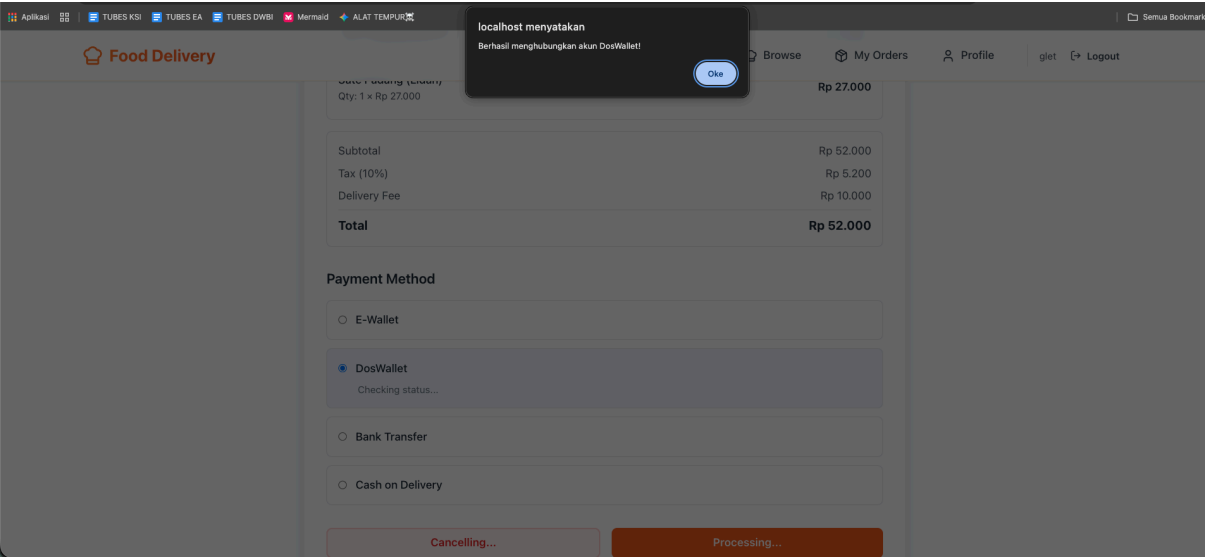
Akun DosWallet belum terhubung.
Hubungkan Akun

☐ Bank Transfer

☐ Cash on Delivery

Cancel Order

Confirm Payment



9.3.3 Visualisasi Saldo Real-time dan Eksekusi Pembayaran

Setelah akun terhubung, antarmuka pengguna menampilkan informasi saldo *DosWallet* secara *real-time*. Data ini diambil melalui *query* GraphQL sesaat setelah halaman dimuat, memastikan pengguna melihat saldo aktual sebelum bertransaksi.

Sistem juga menerapkan validasi saldo di sisi klien (*Client-side Validation*). Jika saldo *DosWallet* lebih kecil dari total tagihan pesanan, tombol "*Pay Now*" akan dinonaktifkan (disabled) atau menampilkan pesan peringatan, mencegah pengguna melakukan transaksi yang pasti gagal. Eksekusi pembayaran dilakukan dengan memanggil mutasi GraphQL *payOrder* tanpa perlu mengirimkan nominal harga secara manual dari *frontend*, melainkan hanya mengirimkan *orderId* dan metode pembayaran, menutup celah manipulasi harga di sisi klien.

Food Delivery

Browse My Orders Profile Customer One Logout

Items

Sate Padang (Daging) Qty: 2 x Rp 25,000	Rp 50,000
--	-----------

Subtotal Rp 50,000
Tax (10%) Rp 5,000
Delivery Fee Rp 10,000
Total **Rp 50,000**

Payment Method

☐ E-Wallet

☒ **DosWallet** Saldo: Rp 2,431,000
Saldo mencukupi. Siap bayar.

☐ Bank Transfer

☐ Cash on Delivery

Cancel Order Confirm Payment

BAB 10 TIMELINE Pengerjaan

Kegiatan pengembangan sistem dilaksanakan mengikuti jadwal intensif selama 4 minggu dengan rincian fokus dan target luaran sebagai berikut:

Minggu	Fokus Kegiatan	Output Terukur
Minggu 13 <i>(Core Development)</i>	Pengembangan Core Services & Infrastruktur Docker 1. Finalisasi logika bisnis pada <i>User Service</i> (JWT), <i>Restaurant Service</i>, dan <i>Order Service</i>. 2. Penyusunan skema GraphQL (<i>Schema-First</i>) dan Resolver. 3. Konfigurasi <code>docker-compose.yml</code> agar 3 service utama + database dapat berjalan serentak.	a) Container user, restaurant, order status Up/Healthy. b) Login berhasil menghasilkan Token JWT. c) Query List Restoran & Menu berjalan di Apollo Sandbox. d) Mutasi <code>createOrder</code> berhasil mencatat data (status mock).
Minggu 14 <i>(Integration & Expansion)</i>	Integrasi Lintas Kelompok & Service Pendukung 1. Menghubungkan <i>Order Service</i> dengan API GraphQL DOSWallet (Mitra) untuk pembayaran real. 2. Pengembangan <i>Driver Service</i> (Alokasi pengemudi) dan <i>Payment Service</i> (Pencatatan Log). 3. Implementasi mekanisme <i>Error Handling</i> jika saldo mitra kurang atau API mati.	a) Transaksi Order memotong saldo di DOSWallet (Integrasi Terbukti). b) Status Order otomatis berubah PAID -> FINDING_DRIVER. c) <i>Driver Service</i> berhasil meng-assign pengemudi ke order aktif.

Minggu 15 <i>(Testing & Doc)</i>	Pengujian Sistem & Penyusunan Laporan 1. <i>End-to-End Testing</i> skenario lengkap: Login -> Pilih Menu -> Bayar (External) -> Dapat Driver. 2. Validasi keamanan (Cek Token Expired, Role Access). 3. Finalisasi Laporan Dokumen (Bab 1 s.d. Lampiran).	a) Screenshot bukti pengujian lengkap. b) Dokumen Proposal/Laporan Final siap 100%. c) Video demo singkat alur aplikasi (opsional).
Minggu 16 <i>(Finalization)</i>	Final Review & Presentasi 1. <i>Code Freeze</i> (Tidak ada perubahan fitur mayor). 2. Pengecekan deployment akhir (memastikan docker-compose up lancar saat demo). 3. Pembuatan Slide Presentasi (PPT).	a) Slide PPT Presentasi. b) Repository GitHub rapi (README.md lengkap). c) Demo Sistem Berjalan di hadapan dosen.

BAB 11 DELIVERABLES

Sesuai dengan ketentuan Tugas Besar, tim kami akan menyerahkan artefak proyek sebagai berikut:

11.1 Dokumentasi Proyek (Laporan PDF)

Dokumen komprehensif yang mencakup:

1. Arsitektur Sistem: Diagram Microservices, ERD Database per Service, dan Topologi Docker.
2. Kontrak API: Dokumentasi Skema GraphQL (typeDefs) untuk setiap service beserta contoh *Query* dan *Mutation* utama.
3. Bukti Integrasi: Penjelasan teknis dan *screenshot* log komunikasi antara *Order Service* internal dengan *DOSWallet* eksternal (Bukti sebagai Consumer).
4. Dokumentasi Provider: Bukti endpoint profil user yang dibuka untuk kelompok lain (Bukti sebagai Provider).

11.2 Kode Program (GitHub Repository)

Repository terstruktur dengan organisasi folder yang rapi:

1. /user-service: Layanan Identitas & Autentikasi.
2. /restaurant-service: Layanan Katalog Menu.
3. /order-service: Layanan Transaksi Utama.
4. /payment-service: Layanan Wrapper Pembayaran.
5. /driver-service: Layanan Manajemen Armada.
6. docker-compose.yml: File orkestrasi utama untuk menjalankan seluruh sistem dengan satu perintah.
7. README.md: Panduan instalasi dan cara menjalankan sistem (*Run Guide*).

11.3 Bahan Presentasi (PPT)

Slide presentasi yang memuat:

1. Latar belakang dan solusi arsitektur.
2. Skenario Demo: Login User → Browsing Menu → Checkout Order → Validasi Pembayaran ke Mitra → Penugasan Driver.
3. Penjelasan pembagian peran dan tantangan teknis yang diselesaikan.

BAB 12 KRITERIA KEBERHASILAN

Proyek ini dianggap berhasil dan memenuhi standar penilaian jika:

1. Full Dockerization: Kelima layanan (*User, Restaurant, Order, Payment, Driver*) dan lima database-nya dapat dijalankan secara bersamaan menggunakan docker-compose up tanpa error dependensi.
2. GraphQL Compliance: Seluruh komunikasi antara klien dan server menggunakan GraphQL. Schema terdefinisi dengan jelas dan dapat diuji melalui *Apollo Sandbox* atau *GraphQL Playground*.
3. Proven Interoperability: Terjadi pertukaran data nyata antara *Order Service* dengan *Payment Service Eksternal* (DOSWallet).
 - a. Jika Saldo DOSWallet Cukup → Order Berhasil (PAID).
 - b. Jika Saldo DOSWallet Kurang → Order Gagal (CANCELLED).
4. Reproducibility: Dosen atau Asisten Laboratorium dapat men-*clone* repository dan menjalankan sistem di mesin lokal mereka dengan mengikuti panduan dokumentasi.